

Correction TP 1 : Jeux de données et variables

Exercice 0 : Prise en main

J'aime utiliser **Rstudio**, une interface du logiciel **R** qui est bien pratique car elle comprend :

- Une fenêtre contenant notre script **R**
- Une fenêtre console pour exécuter des commandes (comme dans **R**)
- Une fenêtre contenant les onglets **Environment** (environnement)
- Une fenêtre contenant les onglets **Plots** (graphiques), **Packages**, **Help** (aide)

Pour installer **Rstudio** c'est ici

R Markdown permet de créer un document avec du texte, des extraits de code **R** et leurs résultats, et d'obtenir un beau PDF. Plus d'informations ici, et une cheat sheet bien pratique ici

Si vous préférez vous pouvez aussi utiliser **Sweave**

1. Installer un package

Le Comprehensive R Archive Network (CRAN) est le répertoire officiel de packages **R**, et c'est le répertoire par défaut dans **Rstudio**. Avant de pouvoir utiliser un package, il faut s'assurer que le package soit installé sur notre machine

Pour installer un package, on peut utiliser la commande `install.packages` ; dans ce cas, faire attention aux guillemets. Sinon, on peut cliquer sur l'onglet **Packages** de la fenêtre en bas à droite (dans **Rstudio**), puis indiquer le nom du package souhaité dans le prompt

Par exemple, avec la ligne de code ci-dessous on installe les packages **tinytex** et **rmarkdown**

```
install.packages("tinytex")
install.packages("rmarkdown")
```

À faire : Installer les packages **faraway** que l'on trouve dans CRAN

```
install.packages("faraway")
```

Note : on ne peut pas run de `install.packages` dans un **Rmarkdown**, c'est pourquoi dans les chunks ci-dessus on utilise `{r eval=FALSE}`

2. Charger un package

Une fois qu'un package est installé, il faut le charger avant de pouvoir l'utiliser. La commande `library` permet de faire cela

Par exemple, avec la ligne de code ci-dessous on charge le package **faraway**

```
library(faraway)
```

À faire : Installer et charger le package **ggplot2**

```
install.packages("ggplot2")
```

```
library(ggplot2)
```

3. Documentation du package

On accède à la documentation du package en cliquant sur son nom dans l'onglet Packages de la fenêtre en bas à droite. De là, on peut ensuite accéder à la documentation de chacune des fonctions ou jeux de données contenus dans le package

On peut aussi accéder directement à la documentation grâce à la commande ?

Par exemple, avec la ligne de code ci-dessous on accède à la documentation de **pima**. C'est un jeu de données contenu dans le package **faraway**

```
?pima
```

À faire : Accéder à la documentation d'un jeu de données contenu dans le package faraway à partir de l'onglet Packages. Le nom du jeu de données est (par exemple) **beetle**

4. Variables

On aime commencer avec un espace de travail vide

```
rm(list = ls())
```

Les variables sont utilisées pour stocker des données

Une variable peut avoir un nom court (par exemple **x** ou **y**) ou un nom plus descriptif (par exemple que **bmi** ou **body_mass_index**). Mais :

- Le nom d'une variable ne peut pas commencer par un chiffre ; il doit commencer par une lettre
- Le nom d'une variable peut seulement contenir des caractères alphanumériques et des underscores (par exemple **az9_8**)
- Le nom d'une variable est sensible aux majuscules et minuscules (par exemple **az9_8**, **Az9_8**, et **AZ9_8** sont différentes variables)

! Attention à ne pas utiliser un nom déjà utilisé dans des packages pour nommer une fonction ou un jeu de données

On utilise **=** pour assigner la valeur à la variable

Par exemple, avec la ligne de code ci-dessous on assigne la valeur 7 à la variable nommée **az9_8**

```
az9_8 = 7
```

On retrouve l'ensemble des objets stockés dans l'onglet Environment, avec la commande **ls** (pour list objects)

```
ls()
```

```
## [1] "az9_8"
```

La commande `rm` (pour remove) permet d'effacer un objet qui était stocké

Par exemple, avec la ligne de code ci-dessous on supprime `az9_8`

```
rm(az9_8)
```

5. Data types dans R

En plus du type `numeric` dont le nom est explicite, on a

- `integer` : entier
- `character` : texte ou chaîne de texte
- `logical` : objet binaire (booléen) pouvant seulement avoir les valeurs `TRUE` ou `FALSE`
- `factor` : objet catégoriel (les `levels` sont ses valeurs possibles)

La commande `class` permet de connaître le type d'un objet, et on peut utiliser des commandes telles que `is.numeric`, `is.character`, `is.logical` pour savoir si un objet est de type `numeric` etc.

```
class(2)
```

```
## [1] "numeric"
```

```
class(2L)
```

```
## [1] "integer"
```

```
class("texte")
```

```
## [1] "character"
```

```
class(TRUE)
```

```
## [1] "logical"
```

Par exemple, la commande `is.factor` permet de savoir si un objet est de type `factor`

`NULL` est l'objet vide

La commande `levels` permet d'obtenir le nom des différentes catégories d'un objet catégoriel

6. Operations

Question : Que fait la ligne de code ci-dessous ? *Réponse* : Elle génère la séquence de nombres de 1 à 50

```
1:50
```

```
## [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
```

Pour faire des opérations sur des variables :

- Opérateurs élémentaires d'arithmétique : `+`, `-`, `*`, `/`, `^`, etc.

```
1 + 2
3 * 6
```

- Opérateur pour générer une séquence : :

```
200:209
```

```
## [1] 200 201 202 203 204 205 206 207 208 209
```

- Opérateurs d'assignation : =

```
x = 2
y = (-1)^2
z = 10 / 3
w = "lapin" # Les guillemets de part et d'autre du texte indique que w est de type character
class(w)
```

```
## [1] "character"
```

- Opérateurs de comparaison : ==, !=, >, <, >=, <=

```
x > y
```

Question : Que fait que fait l'opérateur ci-dessous ? Réponse : Il teste si x et y sont égaux

```
x != y
```

```
## [1] TRUE
```

- Opérateurs de logique : &, |, !
- Opérateur pour tester une condition et exécuter une instruction si elle est vérifiée : if

```
if(z < 2){ z = "petit" }else{ z = "grand" }
z
```

```
## [1] "grand"
```

```
if( (x > 1) | (y == 2) ){ print(z) }
```

```
## [1] "grand"
```

- Opérateurs pour écrire une boucle : for, while

```
for(i in 1:3){ x = x + 1 }

while(y > 0.1){ y = y / 2 }
```

- Opérateur pour accéder à des éléments d'un vecteur, d'une matrice, d'une liste ou d'un data frame : []
- Opérateur pour accéder à des éléments d'une liste ou d'un data frame : \$
- Et beaucoup d'autres

7. Vecteurs

Les vecteurs regroupent des éléments du même type sous forme de séquence. On peut utiliser la commande `c` pour créer un vecteur

Par exemple, le vecteur ci-dessous contient les éléments 1, 2 et 4

```
v = c(1, 2, 4)
```

On peut aussi utiliser l'opérateur permettant de générer une séquence. Par exemple, la ligne de code ci-dessous crée vecteur `w` contenant les éléments de 100 à 150

```
w = 100:150
```

Avec la ligne de code ci-dessous, le vecteur `z` créé contient 1000 fois le texte "souris"

```
z = rep("souris", times = 1000)
```

Avec la ligne de code ci-dessous, le vecteur `y` créé contient les valeurs de 0 à 1 par incrément de 0.1

```
y = seq(from = 0, to = 1, by = 0.1)
```

On peut accéder à l'un (ou plusieurs) élément d'un vecteur en indiquant son indice entre crochet immédiatement après le nom du vecteur

Par exemple, avec la ligne de code ci-dessous on accède au troisième élément de `v`

```
v[3]
```

```
## [1] 4
```

! Sous 'R', les indices commencent à 1

Avec la ligne de code ci-dessous on accède à tous les éléments de `v`, sauf le deuxième

```
v[-2]
```

```
## [1] 1 4
```

Avec la ligne de code ci-dessous on accède au troisième et dixième, onzième et douzième éléments de `w`

```
w[c(3, 10:12)]
```

```
## [1] 102 109 110 111
```

La commande `length` permet de connaître la longueur d'un vecteur

```
length(w)
```

```
## [1] 51
```

On peut utiliser les commande `as.numeric`, `as.character`, `as.factor`, ou `as.logical` pour potentiellement changer le type d'un objet

Par exemple :

```
as.logical(c(0,1,1))
```

```
## [1] FALSE TRUE TRUE
```

```
as.factor(c(1,2,2,5,1))
```

```
## [1] 1 2 2 5 1
## Levels: 1 2 5
```

Question : Que font que font les lignes de code ci-dessous ? *Réponse :* Le vecteur `v` est de type `character`, on peut le transformer en `factor`, qui aura comme levels "`a`", "`b`", et "`c`"

```
v = c("a", "b", "c", "a", "c", "c")
class(v)
```

```
## [1] "character"
```

```
w = as.factor(v)
class(w)
```

```
## [1] "factor"
```

```
levels(w)
```

```
## [1] "a" "b" "c"
```

```
z = as.character(w)
class(z)
```

```
## [1] "character"
```

8. Quelques fonctions usuelles

Logarithme, exponentielle, fonctions trigonométriques etc.

```
a = log(1)
b = exp(3)

d = sin(pi)
aa = cos(pi)

bb = sqrt(4)
```

Question : Que font les lignes de code ci-dessous ? *Réponse :* Génèrent aléatoirement un échantillon de taille 10 sous la loi Uniforme $U(0,1)$, sous la loi Bernouilli $B(0.5)$, et sous la loi Binomiale $B(5,0.5)$

```
x = runif(n = 10, min = 0, max = 1)

y = rbinom(n = 10, size = 1, prob = 0.5)
z = rbinom(n = 10, size = 5, prob = 0.5)
```

Question : Que font les lignes de code ci-dessous ? *Réponse* : tire aléatoirement et sans remplacement 8 éléments parmi l'ensemble $\{1, \dots, 10\}$, et tire aléatoirement et avec remplacement 8 éléments parmi l'ensemble $\{1, \dots, 10\}$

```
x = sample(x = 1:10, size = 8, replace = FALSE)
y = sample(x = 1:10, size = 8, replace = TRUE)
```

Question : Que font les lignes de code ci-dessous ? *Réponse* : Génère aléatoirement un échantillon de taille 1000 sous la loi Normale $N(5,4)$, calcule la moyenne, la variance, les quartiles, et les déciles de cet échantillon

```
x = rnorm(n = 1000, mean = 5, sd = 2)
mean(x)
```

```
## [1] 4.968329
```

```
var(x)
```

```
## [1] 3.901506
```

```
quantile(x)
```

```
##          0%          25%          50%          75%         100%
## -0.4649669  3.6439990  4.9572822  6.3974082 11.2516234
```

```
quantile(x, probs = seq(0, 1, 0.1))
```

```
##          0%          10%          20%          30%          40%          50%          60%
## -0.4649669  2.4203709  3.2886308  3.8598598  4.4348949  4.9572822  5.4173865
##          70%          80%          90%         100%
##  6.0546207  6.6579464  7.5151491 11.2516234
```

Question : Que font les lignes de code ci-dessous ? *Réponse* : Génère aléatoirement un échantillon de taille 1000 sous la loi Normale $N(5,4)$, et crée une variable catégorielle à partir du tirage avec remplacement de 1000 éléments parmi l'ensemble $\{1,2,3\}$ et avec probabilités 0.2, 0.5, et 0.3

```
x = rnorm(n = 1000, mean = 5, sd = 2)
v = as.factor(sample(1:3, size = 1000, replace = TRUE, prob = c(0.2, 0.5, 0.3)))
```

Question : Que font les lignes de code ci-dessous ? *Réponse* : résumés statistiques d'une variable qualitative (tables de contingence en effectif ou proportion) et d'une variable quantitative

```
table(v)
```

```
## v
##  1  2  3
## 219 502 279
```

```
prop.table(table(v))
```

```
## v
##      1      2      3
## 0.219 0.502 0.279
```

```
summary(v)
```

```
##      1      2      3
## 219 502 279
```

```
summary(x)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.3829  3.7192  5.0884  5.0770  6.3814 12.0467
```

Question : Que font les lignes de code ci-dessous ? *Réponse :* classe les éléments du vecteur v par ordre croissant ou décroissant

```
v = c(1,10,3,8,-5)
sort(v)
```

```
## [1] -5  1  3  8 10
```

```
sort(v, decreasing = TRUE)
```

```
## [1] 10  8  3  1 -5
```

Question : Que fait la ligne de code ci-dessous ? *Réponse :* donne la permutation des éléments de v lorsque l'on veut les ordonner de manière croissante

```
order(v)
```

```
## [1] 5 1 3 4 2
```

8. Matrice

La commande `matrix` permet de créer une matrice

```
M = matrix(c(1,2,3,4,5,6), nrow = 2, ncol = 3, byrow = TRUE,
           dimnames = list(c("row1", "row2"), c("col1", "col2", "col3")))
P = matrix(c(1,2,3,4,5,6), nrow = 2, ncol = 3, byrow = FALSE)
```

Question : Que font les lignes de code ci-dessous ? *Réponse :* $M_{\{1,2\}}$, 1e et 2e colonne de P , indique si M est une matrice


```
M[1,2]
```

```
## [1] 2
```

```
P[,1:2]
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

```
is.matrix(M)
```

```
## [1] TRUE
```

Question : Que font les lignes de code ci-dessous ? *Réponse* : nombre de lignes de M, nombre de colonnes de M

```
nrow(M)
```

```
## [1] 2
```

```
ncol(M)
```

```
## [1] 3
```

Question : Que fait la commande `t` ? *Réponse* : La commande `t` permet d'obtenir la transposée d'une matrice

```
t(M)
```

```
##      row1 row2
## col1    1    4
## col2    2    5
## col3    3    6
```

```
t(M) %*% P
```

```
##      [,1] [,2] [,3]
## col1    9   19   29
## col2   12   26   40
## col3   15   33   51
```

La commande `solve` permet d'inverser une matrice (invertible)

```
M = matrix(c(1,2,3,4), nrow = 2, ncol = 2)
solve(M)
```

```
##      [,1] [,2]
## [1,]  -2  1.5
## [2,]   1 -0.5
```

La commande `diag` permet d'extraire les éléments diagonaux d'une matrice

```
diag(M)
```

```
## [1] 1 4
```

La commande `diag` permet aussi de créer une matrice diagonale à partir d'un vecteur

```
diag(1:23)
```

Exercice : Créer une matrice carrée de type `numeric` et de taille 10, et vérifier que le produit de cette matrice par son inverse donne la matrice identité

```
M = matrix(rnorm(100), nrow = 10, ncol = 10)
M %*% solve(M)
```

Exercice : Créer une matrice carrée nommée `P` de taille 5 dont tous les coefficients sont obtenus à partir de tirages Bernoulli avec probabilité de succès $p = 0.6$. Créer ensuite la matrice diagonale `Q` contenant les éléments diagonaux de `P`

```
P = matrix(rbinom(n = 25, size = 1, prob = 0.6), nrow = 5, ncol = 5)
Q = diag(diag(P))
```

9. Listes

La commande `list` permet de créer une liste

```
l = list(1:10, c("lapin", "souris"))
l[[2]]
```

```
## [1] "lapin" "souris"
```

```
l[[c(1,7)]]
```

```
## [1] 7
```

```
l = list(a = 1:10, b = c("lapin", "souris"))
l$a
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

La commande `is.list` indique si un objet est une liste

```
is.list(l)
```

```
## [1] TRUE
```

10. Données manquantes

Les valeurs manquantes sont (généralement) indiquées par `NA`, pour Not Available

```
x = c(1,2,3,NA)
```

La commande `is.na` indique si une valeur est manquante

```
is.na(x)
```

```
## [1] FALSE FALSE FALSE  TRUE
```

! NaN, qui veut dire Not a Number, indique une valeur impossible, comme par exemple un forme indéterminée 0/0

! Inf and -Inf indique + l'infini et - l'infini

11. Écrire une fonction

La commande `function` permet de définir une fonction

```
f = function(x){
  x + 2
}
f(6)
```

```
## [1] 8
```

```
g = function(x, y = 2){
  z = x + y
  if(z > 4){
    x = 9
  }
  return(list(x = x, z = z))
}
g(x = 1, y = 2)
```

```
## $x
## [1] 1
##
## $z
## [1] 3
```

```
g(x = 1, y = 5)
```

```
## $x
## [1] 9
##
## $z
## [1] 6
```

```
res = g(x = 1, y = 5) # l'objet retourné par la fonction g est une list
res$x
```

```
## [1] 9
```

Question : Que se passe-t-il avec la ligne de code ci-dessous ? *Réponse :* on calcule $g(x = 1, y = 2)$, car la valeur par défaut de y est 2

```
g(x = 1)
```

```
## $x
## [1] 1
##
## $z
## [1] 3
```

12. Structure classique de jeux de données

Un data frame est une collection de variables ayant toutes le même nombre d'observations. Ces variables sont rangées en colonnes, et la structure du data frame est assez similaire à celle d'une matrice. Les colonnes du data frame peuvent être de types différents

```
char = c("a", "b", "d", rep("f", 3), "s", rep("r", 3))
x = rep(1, times = 10)
y = 1:10
d = data.frame(x = x, y = y, char = char)
```

Question : Que font les lignes de code ci-dessous ? *Réponse :* Liste des noms des colonnes de `d`, extrait la variable `char` de `d`, valeur de la 3e variable de `d` chez le premier individu

```
colnames(d)
```

```
## [1] "x"    "y"    "char"
```

```
d$char
```

```
## [1] "a" "b" "d" "f" "f" "f" "s" "r" "r" "r"
```

```
d[1,3]
```

```
## [1] "a"
```

13. Charger un jeu de données

De nombreux jeux de données sont disponibles à travers les packages sur CRAN. Par exemple le package `faraway` contient le jeu de données `pima`

Question : Que fait la ligne de code ci-dessous ? *Réponse :* on charge le jeu de donnée `pima` et le stocke dans la variable nommée `data`

```
data = pima
```

On peut aussi charger un jeu de données depuis un fichier sur notre ordinateur ou un site internet (url). La commande à utiliser dépend du format du fichier, mais en général ce seront des fichiers au format .csv, et on utilisera donc la commande `read.csv`

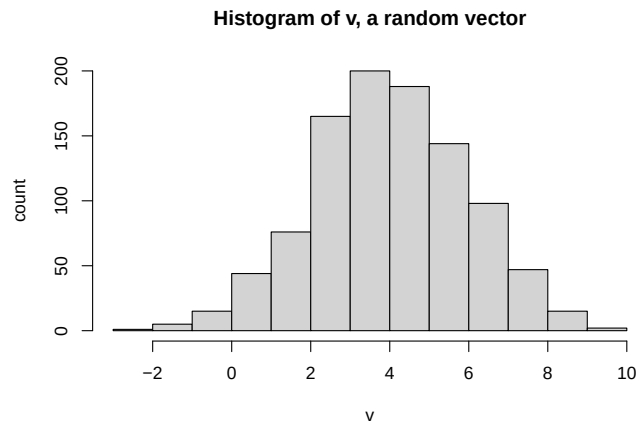
Le premier argument de cette fonction est `file`. Il faut y spécifier le chemin vers votre fichier (ou le nom du fichier s'il est dans le dossier que vous utilisez déjà comme working directory), ou l'url

```
data = read.csv("../titanic.csv")
```

15. Quelques fonctions graphiques

Pour tracer un histogramme on utilise la commande `hist`

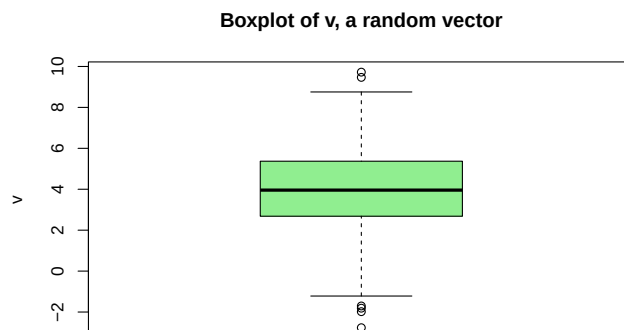
```
v = rnorm(1000, mean = 4, sd = 2)
hist(v, main = "Histogram of v, a random vector", xlab = "v", ylab = "count")
```



La commande `main` permet de spécifier le titre du graphique, `xlab` le nom de l'axe des abscisses, et `ylab` celui des ordonnées

Pour tracer un boxplot on utilise la commande `boxplot`

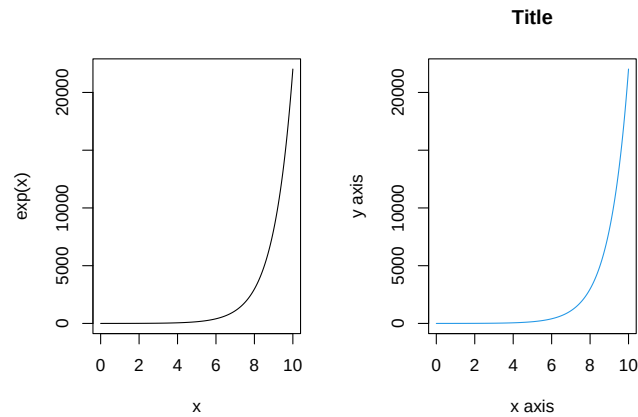
```
boxplot(v, col = 'lightgreen', main = "Boxplot of v, a random vector", ylab = "v")
```



La commande `col` permet de spécifier la couleur voulue pour le graphique

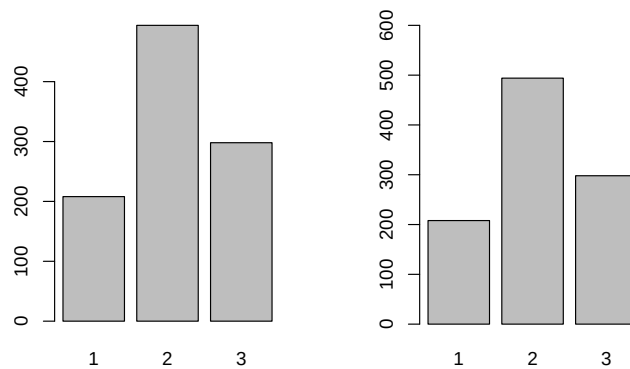
Question : Que fait la ligne de code `par(mfrow = c(1,2))` ci-dessous ? *Réponse :* Elle permettra de placer les graphiques dans un tableau à une ligne et deux colonnes (c'est à dire cote à cote)

```
x = seq(from = 0, to = 10, length.out = 1000)
par(mfrow = c(1,2))
plot(x, exp(x), type = 'l')
plot(x, exp(x), type = 'l', col = '4', main = "Title",
      xlab = "x axis", ylab = "y axis")
```



Question : Que fait l'argument ylim ci-dessous ? *Réponse :* Spécifie les valeurs minimale et maximale souhaitée pour l'axe des ordonnées

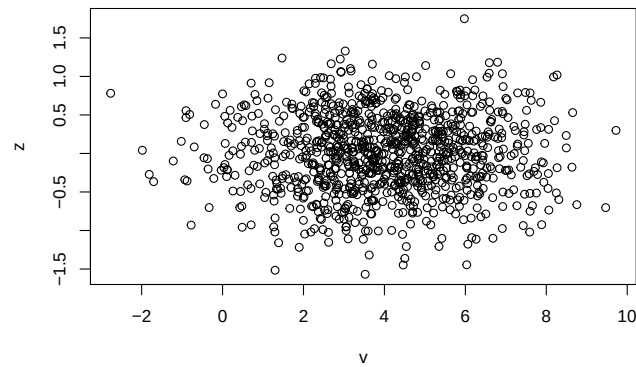
```
w = as.factor(sample(1:3, size = 1000, replace = TRUE, prob = c(0.2, 0.5, 0.3)))
par(mfrow = c(1,2))
barplot(table(w))
barplot(table(w), ylim = c(0,600))
```



Lorsque l'on veut tracer un barplot on doit spécifier le nombre d'observations dans chacune des catégories. La commande `table` ou `summary` nous donne cette information

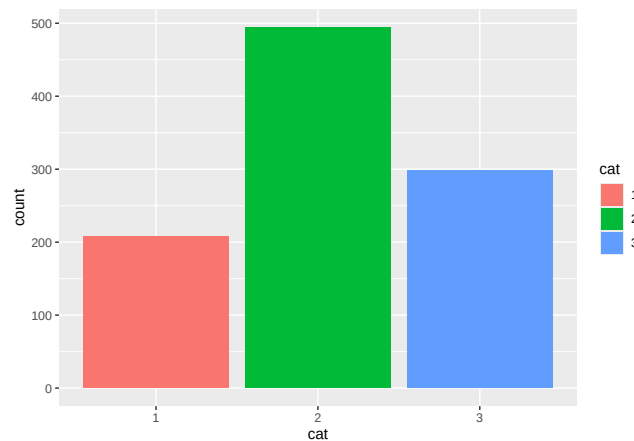
Question : Comment s'appelle le graphique obtenu ci-dessous ? *Réponse :* Un nuage de points

```
z = rnorm(1000, mean = 0, sd = 0.5)
plot(v,z)
```



Le package `ggplot2` peut aussi créer de tels graphes, mais avec plus d'options graphiques (et de manière plus élégante). Par exemple

```
w = as.data.frame(w)
names(w) = "cat"
ggplot(w, aes(x = cat, fill = cat)) + geom_bar( )
```



Exercice 1 : Analyse préliminaire d'un jeu de données

1. Charger le jeu de donnée pima

Le jeu donnée est disponible dans le package `faraway`

```
library(faraway)
data(pima)
```

Question : Que fait la ligne de code ci-dessous ? *Réponse* : Elle imprime les 6 premières lignes du data frame

```
head(pima)
```

```
##   pregnant glucose diastolic triceps insulin  bmi diabetes age test
## 1         6     148         72      35        0 33.6   0.627  50    1
## 2         1      85         66      29        0 26.6   0.351  31    0
```

```
## 3      8      183      64      0      0 23.3      0.672 32      1
## 4      1      89      66     23     94 28.1      0.167 21      0
## 5      0     137     40     35    168 43.1      2.288 33      1
## 6      5     116     74      0      0 25.6      0.201 30      0
```

Question : Quel est le nombre de lignes dans le jeu de données ? Le nombre de colonnes ? *Réponse :* 768 lignes et 9 colonnes

```
nrow(pima)
```

```
## [1] 768
```

```
ncol(pima)
```

```
## [1] 9
```

Dans le jeu de donnée `pima` il y a donc 768 individus et 9 variables

Question : En utilisant ce qui a été fait précédemment, imprimer la liste des noms de toutes les variables. À l'aide de la documentation disponible, donner une description courte de chaque variable, et indiquer lesquelles devraient être qualitatives et lesquelles devraient être quantitatives

```
colnames(pima)
```

```
## [1] "pregnant" "glucose"   "diastolic" "triceps"   "insulin"   "bmi"
## [7] "diabetes"  "age"       "test"
```

```
?pima
```

Description :

- ...
- ...

Question : Quelle ligne de code permet d'accéder à la variable `bmi` ? *Réponse :*

```
pima$bmi
```

Très souvent, la variable `age` est catégorisée (pour éventuellement atténuer l'effet de potentielles valeurs extrêmes, proposer un moyen de gérer les données manquantes, etc.). Dans ce cas il faut choisir les catégories. Nous avons plusieurs possibilités ; lesquelles ? Coder l'option choisie

Réponse :

- Catégories basées sur des intervalles d'âge de même largeur

```
min(pima$age, na.rm = TRUE) # 21
```

```
## [1] 21
```



```
max(pima$age, na.rm = TRUE) # 81
```

```
## [1] 81
```

```
pima$age_cat = cut(pima$age,
                   breaks = seq(from = 21, to = 81, length.out = 5),
                   include.lowest = TRUE) # 4 intervalles
```

Mais on pourrait se retrouver avec des catégories contenant très peu de femmes

- Catégories basées sur des intervalles d'âge contenant le même nombre de femmes

```
pima$age_cat = cut(pima$age,
                   breaks = round(unname(quantile(pima$age, na.rm = TRUE))),
                   include.lowest = TRUE) # 4 intervalles
```

J'ai arrondi les bornes des intervalles, c'est pourquoi on a à peu près le même nombre de femmes dans chaque catégorie

- Catégories basée sur des groupes d'âges qui font sens, comme par exemple enfants (moins de 13 ans), adolescents (entre 13 et 18 ans), adultes (entre 18 et 60 ans), et seniors (plus de 60 ans), ou mineurs, adultes, et seniors. La définition de ces groupes dépend de l'application (par exemple en santé, de la maladie d'intérêt)

```
pima$age_cat = cut(pima$age,
                   breaks = c(21, 30, 55, 65, 81),
                   include.lowest = TRUE)
```

2. Analyse statistique univariée

Question : Quelles quantités calcule-t-on pour l'analyse statistique descriptive d'une variable quantitative ?
Quelles commandes R peut-on utiliser pour cela ? Réponse : La moyenne, la variances, les quartiles

Par exemple, pour la variable `glucose`:

```
mean(pima$glucose)
```

```
## [1] 120.8945
```

```
var(pima$glucose)
```

```
## [1] 1022.248
```

```
quantile(pima$glucose)
```

```
##      0%      25%      50%      75%     100%
##  0.00  99.00 117.00 140.25 199.00
```

Question : Et pour une variable qualitative ? *Réponse* : On compte le nombre d'observations tombées dans chacune des catégories

Par exemple, pour la variable `age_cat`:

```
table(pima$age_cat)
```

```
##
## [21,30] (30,55] (55,65] (65,81]
##      417      301       37       13
```

```
prop.table(table(pima$age_cat))
```

```
##
##      [21,30]      (30,55]      (55,65]      (65,81]
## 0.54296875 0.39192708 0.04817708 0.01692708
```

Sinon on peut réaliser directement une telle analyse à l'aide de la commande `summary`

```
summary(pima)
```

```
##      pregnant      glucose      diastolic      triceps
## Min.   : 0.000   Min.    : 0.0   Min.    : 0.00   Min.    : 0.00
## 1st Qu.: 1.000   1st Qu.: 99.0   1st Qu.: 62.00   1st Qu.: 0.00
## Median : 3.000   Median :117.0   Median : 72.00   Median :23.00
## Mean   : 3.845   Mean    :120.9   Mean    : 69.11   Mean    :20.54
## 3rd Qu.: 6.000   3rd Qu.:140.2   3rd Qu.: 80.00   3rd Qu.:32.00
## Max.    :17.000   Max.    :199.0   Max.    :122.00   Max.    :99.00
##      insulin      bmi      diabetes      age
## Min.    : 0.0   Min.    : 0.00   Min.    :0.0780   Min.    :21.00
## 1st Qu.: 0.0   1st Qu.:27.30   1st Qu.:0.2437   1st Qu.:24.00
## Median : 30.5   Median :32.00   Median :0.3725   Median :29.00
## Mean    : 79.8   Mean    :31.99   Mean    :0.4719   Mean    :33.24
## 3rd Qu.:127.2   3rd Qu.:36.60   3rd Qu.:0.6262   3rd Qu.:41.00
## Max.    :846.0   Max.    :67.10   Max.    :2.4200   Max.    :81.00
##      test      age_cat
## Min.    :0.000   [21,30]:417
## 1st Qu.:0.000   (30,55]:301
## Median :0.000   (55,65]: 37
## Mean    :0.349   (65,81]: 13
## 3rd Qu.:1.000
## Max.    :1.000
```

Question : Il y a-t-il quelque chose d'inattendu ?

Indices : Quel devrait être le type d'une variable quantitative ? Quel devrait être le type d'une variable qualitative ?

Réponse : Une variable quantitative devrait être de type `numeric`, et une variable catégorielle de type `factor`
`test` est une variable binaire, mais à la vue du résumé elle est considérée par R comme une variable quantitative

Ensuite, on regarde les valeurs maximales et minimales de chaque variable : `max(pregnant) = 17` -> surprenant, mais pas impossible ; par contre `blood pressure = 0` -> la personne devrait être morte...

```
sort(pima$diastolic)
```

On a 36 valeurs qui sont égales à 0. Il semblerait que la valeur 0 ait été utilisée pour remplir les valeurs manquantes dans les variables `diastolic`, `glucose`, `triceps`, `insulin`, et `bmi`

À faire : Effectuer des modifications sur le jeu de données en fonction des remarques faites à la question précédente

On indique à R que la variable `test` est binaire

```
pima$test      = as.factor(pima$test)
levels(pima$test) = c("négatif", "positif")
```

On remplace les valeurs manquantes par des NA

```
pima$diastolic[pima$diastolic == 0] = NA
pima$glucose[pima$glucose == 0]    = NA
pima$triceps[pima$triceps == 0]    = NA
pima$insulin[pima$insulin == 0]    = NA
pima$bmi[pima$bmi == 0]            = NA
```

À faire : Refaire l'analyse statistique descriptive des variables

```
summary(pima)
```

```
##      pregnant      glucose      diastolic      triceps
##  Min.   : 0.000   Min.   : 44.0   Min.   : 24.00   Min.   : 7.00
##  1st Qu.: 1.000   1st Qu.: 99.0   1st Qu.: 64.00   1st Qu.:22.00
##  Median : 3.000   Median :117.0   Median : 72.00   Median :29.00
##  Mean   : 3.845   Mean   :121.7   Mean   : 72.41   Mean   :29.15
##  3rd Qu.: 6.000   3rd Qu.:141.0   3rd Qu.: 80.00   3rd Qu.:36.00
##  Max.   :17.000   Max.   :199.0   Max.   :122.00   Max.   :99.00
##                      NA's   :5      NA's   :35      NA's   :227
##      insulin      bmi      diabetes      age
##  Min.   : 14.00   Min.   :18.20   Min.   :0.0780   Min.   :21.00
##  1st Qu.: 76.25   1st Qu.:27.50   1st Qu.:0.2437   1st Qu.:24.00
##  Median :125.00   Median :32.30   Median :0.3725   Median :29.00
##  Mean   :155.55   Mean   :32.46   Mean   :0.4719   Mean   :33.24
##  3rd Qu.:190.00   3rd Qu.:36.60   3rd Qu.:0.6262   3rd Qu.:41.00
##  Max.   :846.00   Max.   :67.10   Max.   :2.4200   Max.   :81.00
##  NA's   :374     NA's   :11
##      test      age_cat
##  négatif:500   [21,30]:417
##  positif:268   (30,55]:301
##                (55,65]: 37
##                (65,81]: 13
##
##
##
```

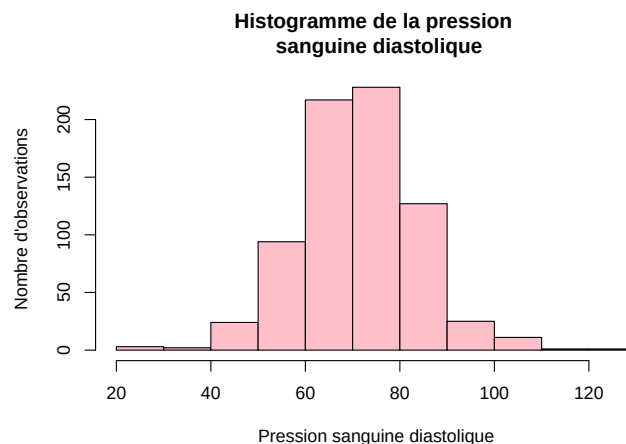
3. Analyse graphique univariée

Question : Quelles représentations graphiques peut-on utiliser pour décrire une variable quantitative ?
Quelles commandes R peut-on utiliser pour cela ? Réponse : Un histogramme ou un boxplot, on peut utiliser les commandes `hist` et `boxplot`, ou la fonction `ggplot` du package `ggplot2`

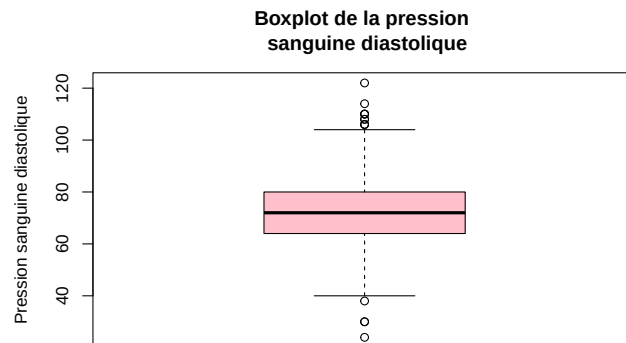
Question : Et pour une variable qualitative ? *Réponse* : Un barplot

À faire : Considérons la pression sanguine diastolique. Représenter son histogramme et son boxplot

```
hist(pima$diastolic, col = "pink", main = "Histogramme de la pression \n sanguine diastolique",
     xlab = "Pression sanguine diastolique", ylab = "Nombre d'observations")
```



```
boxplot(pima$diastolic, col = "pink", main = "Boxplot de la pression \n sanguine diastolique",
        ylab = "Pression sanguine diastolique")
```

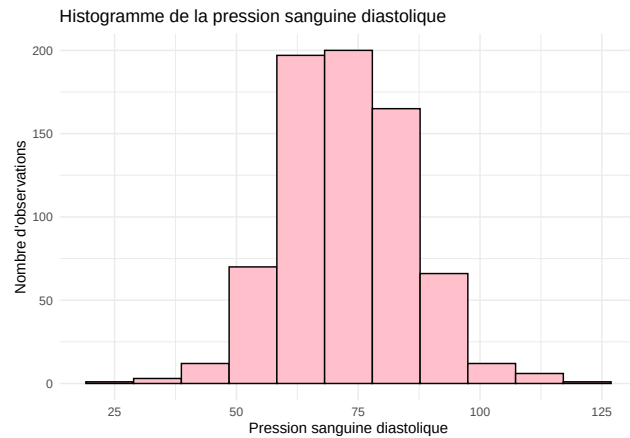


Si l'on souhaite spécifier les blocs/intervalles dans l'historgramme, il faut utiliser l'argument `breaks`

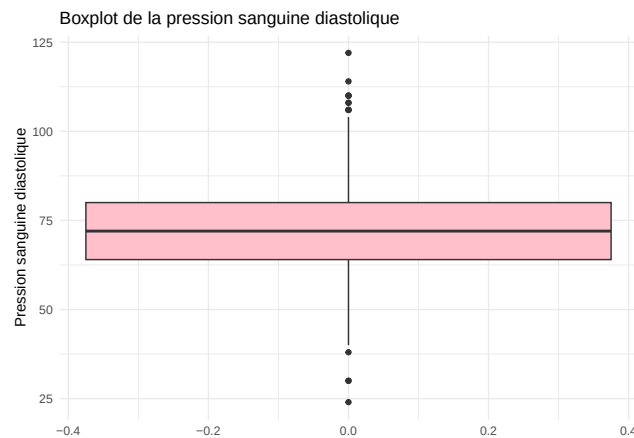
Avec `ggplot` :

```
library(ggplot2)

ggplot(pima, aes(x = diastolic)) +
  geom_histogram(fill = "pink", color = "black", bins = 11, na.rm = TRUE) +
  labs(title = "Histogramme de la pression sanguine diastolique",
       x = "Pression sanguine diastolique", y = "Nombre d'observations") +
  theme_minimal()
```

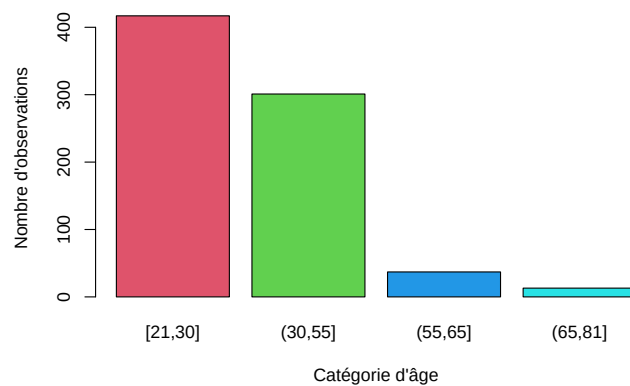


```
ggplot(pima, aes(y = diastolic)) + geom_boxplot(fill = "pink", na.rm = TRUE) +
  labs(title = "Boxplot de la pression sanguine diastolique", y = "Pression sanguine diastolique") +
  theme_minimal()
```



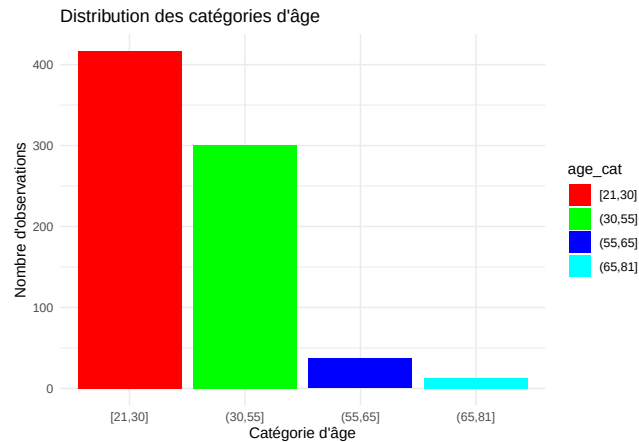
À faire : Considérons l'âge en catégoriel. Représenter son barplot ; on peut utiliser la commande `barplot`

```
barplot(summary(pima$age_cat), col = 2:5, xlab = "Catégorie d'âge",
  ylab = "Nombre d'observations")
```



ou `ggplot` :

```
ggplot(pima, aes(x = age_cat, fill = age_cat)) + geom_bar() +
  scale_fill_manual(values = c("red", "green", "blue", "cyan")) +
  labs(title = "Distribution des catégories d'âge", x = "Catégorie d'âge",
    y = "Nombre d'observations") + theme(legend.position = "none") +
  theme_minimal()
```

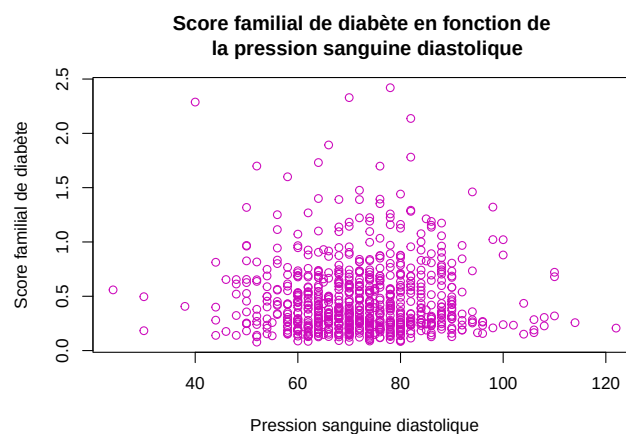


4. Analyse multivariée

Lorsque l'on souhaite représenter graphiquement la relation entre deux variables quantitatives on utilise un nuage de points

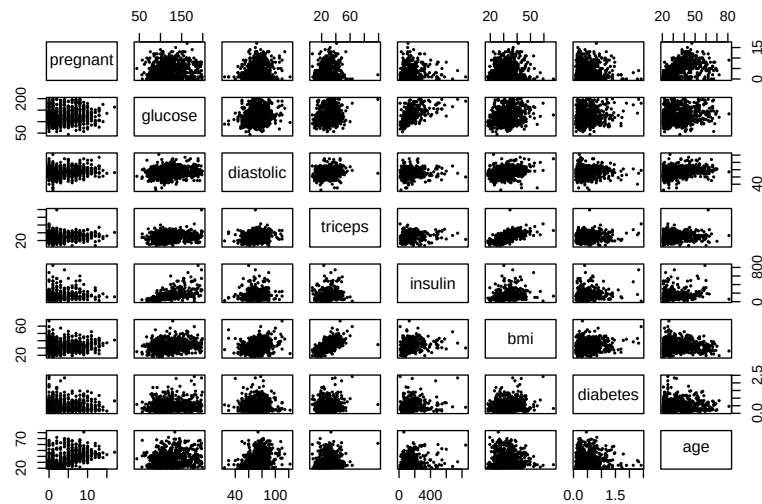
À faire : Représenter graphiquement la relation entre les variables `diabetes` et `diastolic` (diabetes en fonction de diastolic)

```
plot(pima$diastolic, pima$diabetes, col = "6", xlab = "Pression sanguine diastolique",
  ylab = "Score familial de diabète",
  main = "Score familial de diabète en fonction de \n la pression sanguine diastolique")
```



Question : Que fait la ligne de code ci-dessous ? *Réponse :* Les nuages de points des variables quantitatives 2 à 2

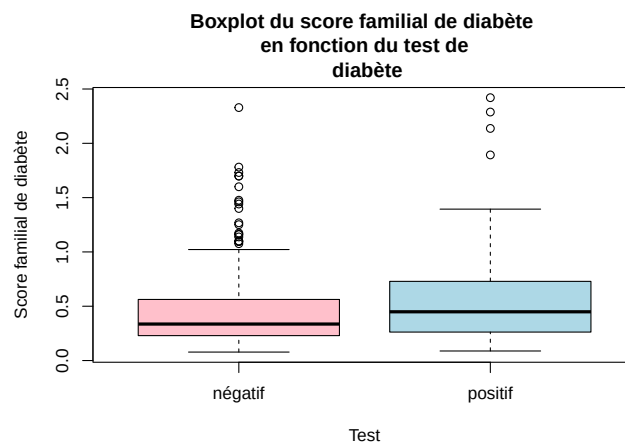
```
pairs(pima[, -c(9,10)], pch = 16, cex = 0.5) # les 9e et 10e variables sont qualitatives
```



Lorsque l'on souhaite représenter graphiquement la relation entre une variable quantitative et une variable qualitative on utilise une boîte à moustaches (boxplot)

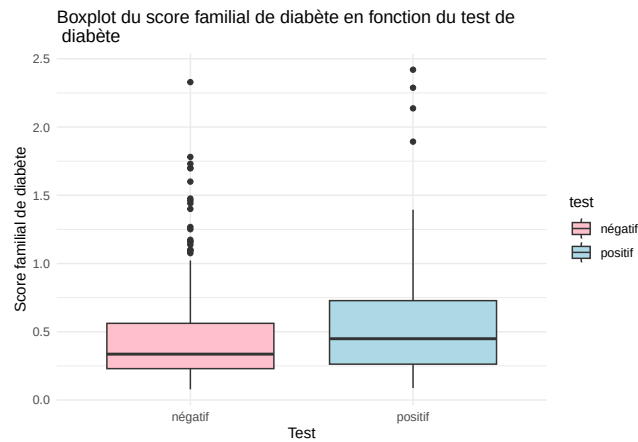
À faire : Représenter graphiquement la relation entre les variables `diabetes` et `test`

```
boxplot(pima$diabetes ~ pima$test, col = c("pink", "lightblue"),
        main = "Boxplot du score familial de diabète \n en fonction du test de \n diabète",
        xlab = "Test", ylab = "Score familial de diabète")
```



Avec ggplot :

```
ggplot(pima, aes(x = test, y = diabetes, fill = test)) + geom_boxplot() +
  scale_fill_manual(values = c("pink", "lightblue")) +
  labs(title = "Boxplot du score familial de diabète en fonction du test de \n diabète",
        x = "Test", y = "Score familial de diabète") + theme(legend.position = "none") +
  theme_minimal()
```



On a un boxplot pour chacun des levels de la variable qualitative. Ici on en a 2 : un pour le test de diabète positif et un pour le test négatif

Lorsque l'on souhaite représenter graphiquement la relation entre deux variable qualitatives on utilise un barplot

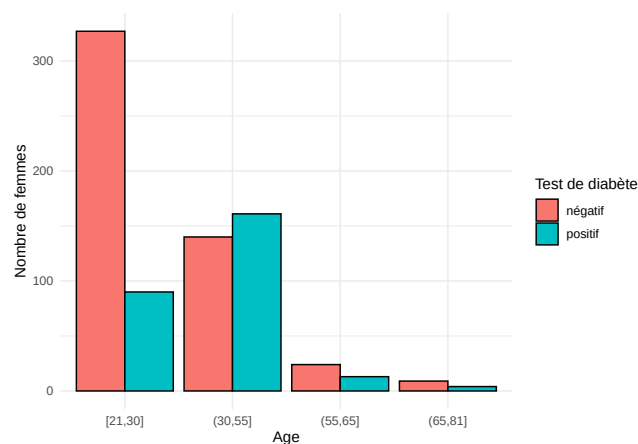
Que font les chunks de code ci-dessous ?

Réponse : On représente graphiquement la relation entre les variables `age_cat` et `tests`

```
install.packages(reshape2)
```

```
# il nous faut le nombre de femmes dans chacune des catégories définies par `age_cat` et `tests`
library(reshape2)
age_test_counts = melt(table(pima$age_cat, pima$test))
colnames(age_test_counts) = c("age_cat", "test", "count")

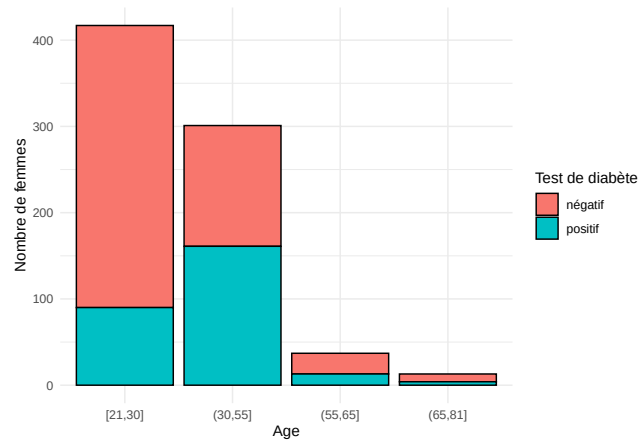
ggplot(data = age_test_counts, aes(x = age_cat, y = count, fill = test)) +
  geom_bar(stat = "identity", color = "black", position = position_dodge()) +
  theme_minimal() + labs(x = "Age", fill = "Test de diabète", y = "Nombre de femmes")
```



On a un barplot pour chacun des levels de la variable `age_cat`

On peut aussi empiler les barres, en retirant l'argument `position = position_dodge()`


```
ggplot(data = age_test_counts, aes(x = age_cat, y = count, fill = test)) +
  geom_bar(stat = "identity", color = "black") +
  theme_minimal() + labs(x = "Age", fill = "Test de diabète", y = "Nombre de femmes")
```



Ou montrer la proportion de femmes ayant un test de diabète positif et négatif dans chacune des catégories d'âge, grace à l'argument `position = "fill"`

```
ggplot(data = age_test_counts, aes(x = age_cat, y = count, fill = test)) +
  geom_bar(stat = "identity", color = "black", position = "fill") +
  theme_minimal() + labs(x = "Age", fill = "Test de diabète", y = "Proportion de femmes")
```

